



# Adding new Application to GCSDK

---

SHAKTI Group | CSE Dept | PS-CDISHA - RISE Lab | IIT Madras

RISE LAB

IIT - Madras

# Makefile

- To compile programs more efficiently the GNU's MAKE utility is used.
- The make utility uses the Makefile to compile programs from source code.
- The output generated by the MAKE utility is in ELF format (with extension as "shakti").

# Steps to add a new Application to GCSDK

- **Create a folder inside the target application**

**example : new application - my\_first\_code , (it is a uart based application)**

**Note\*: Do not add any spaces in between the file name.**

> Home > shakti > gc2025 > GCSDK > software > examples > **uart\_applns**



asm\_int



csr\_test



hello



loopback



switch\_mode



Makefile

New Folder — Dolphin

Create new folder in /home/rj/shakti/gc2025/GCSDK/software/examples/uart\_applns/:

my\_first\_code

✓ OK    ⌂ Cancel



# Steps to add a new Application to GCSDK

- **Create a file with the same name of folder inside the newly created folder.**

**example : my\_first\_code.c**

> Home > shakti > gc2025 > GCSDK > software > examples > uart\_applns > **my\_first\_code**



my\_first\_code.c



# Steps to add a new Application to GCSDK

- **Add a makefile to the newly created folder**
  - 1. Re-use existing Makefile file from another folder within the same application**
  - 2. Write your own custom Makefile.**



> Home > shakti > gc2025 > GCSDK > software > examples > uart\_applns > my\_first\_code



Makefile



my\_first\_code.c

> Home > shakti > gc2025 > GCSDK > software > examples > uart\_applns > hello



hello.c



Makefile





# Steps to add a new Application to GCSDK

- **Open the Makefile in the 'examples' directory.**
  - 1. Add an new 'ifeq' block**
  - 2. Provide path and file name**
  - 3. Add and 'endif' block to close the 'ifeq' block**

```
if [ $(PROGRAM) = my_first_code ]
then
    filepath := uart_applns/my_first_code
else
    @echo "Entry for $(PROGRAM) not present"
endif
```



# Steps to add a new Application to GCSDK

**Make sure the working directory is /GCSDK**  
**Execute the following command**

```
make software PROGRAM=my_first_code TARGET=<target_soc>
```

**The <target\_soc> can either be 'Ganga' or 'Yamuna'.  
\*please note <target\_soc> is case sensitive**

```
make[2]: Leaving directory '/home/rj/shakti/gc2025/GCSDK/software/examples'
cd uart_applns/my_first_code && make my_first_code.riscv TARGET=yamuna DEBUG=
make[2]: Entering directory '/home/rj/shakti/gc2025/GCSDK/software/examples/uart_applns/my_first_code'
/home/rj/belsoc/bsoc_toolchain/build_rv32imac/bin/../../lib/gcc/riscv32-unknown-elf/13.2.0/../../../../../r
d: warning: ./output/my_first_code.shakti has a LOAD segment with RWX permissions
make[2]: Leaving directory '/home/rj/shakti/gc2025/GCSDK/software/examples/uart_applns/my_first_code'
All done !
make[1]: Leaving directory '/home/rj/shakti/gc2025/GCSDK/software/examples'
rj@rj:~/shakti/gc2025/GCSDK$
```



# References

Shakti Documentation: <https://shakti.org.in/documentation.html>

Shakti Blogs: <https://blogshakti.org.in/>

Shakti FPGA files: <https://gitlab.com/shaktiproject/sp2020>

Shakti SDK: <https://gitlab.com/shaktiproject/software/shakti-sdk>

Shakti on Arduino: <https://blogshakti.org.in/how-to-print-hello-world-on-shakti-using-arduino-ide/>

Linux on Shakti: <https://gitlab.com/shaktiproject/software/linux-on-shakti>

Platform IO: <https://registry.platformio.org/platforms/platformio/shakti>

Shakti cores: <https://gitlab.com/shaktiproject/cores>

Shakti peripherals: <https://gitlab.com/shaktiproject/uncore/devices>



# Thank you

